DATABASE MANAGEMENT SYSTEMS LAB

(CSC2131)

PROJECT REPORT

Academic Session: 2026-2027

LIFELINK – BLOOD DONOR AVAILABILITY NETWORK

By:

Pratham Jain (2502051205)
Sireesh Babu .K (2502051198)

Section: I3, B.Tech CSE 3rd Semester



MANIPAL UNIVERSITY
JAIPUR

November 2026

Department of Computer Science and Engineering
Manipal University Jaipur
VPO. Dehmi Kalan, Jaipur, Rajasthan, India – 303007

DECLARATION BY THE STUDENT

I hereby declare that this project LIFELINK – BLOOD DONOR AVAILABILITY NETWORK is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the University or other Institute, except where due acknowledgements has been made in the text.

(Signature)

Place: Jaipur, Rajasthan

Name of Student: Sireesh Babu .K

B.Tech CSE 3rd Semester

TABLE OF CONTENTS

S. No	Contents	Page No.
1	Abstract	3
2	Introduction	4
3	Problem Statement	5
4	Objectives of the Project	6
5	System Requirements (Hardware and Software)	7
6	System Design and Database Schema	8
7	ER Diagram	11
8	Database and Table Creation (DDL Scripts)	12
9	SQL Queries for Data Insertion, Updation, Deletion, and Retrieval	16
10	Front-End Design and Implementation (Case 1 only)	19
11	Results and Screenshots (Verification & Test Matrix)	22
12	Conclusion	25
13	Contributions by Team Members	26
14	References	27

1. ABSTRACT

In emergency medical triage, trauma operations, and acute patient care, immediate access to blood compatible with the recipient's ABO and Rh phenotype is essential. Conventional blood management systems rely on siloed facility inventories and delegate critical clinical eligibility verification—such as donor age restrictions and physiological donation intervals—to vulnerable front-end application code.

LifeLink – Blood Donor Availability Network is an enterprise-grade Relational Database Management System engineered to centralize donor registries, hospital blood requests, and regional blood bank inventories. The system enforces safety policies directly at the database engine level using MySQL BEFORE INSERT and AFTER INSERT triggers (SIGNAL SQLSTATE '45000'). Biological compatibility is stored in a normalized relation rather than hardcoded scripts. An optimized stored procedure executes a 3-way join between emergency requests, the medical compatibility table, and an indexed view of eligible donors to output priority-ranked matching donors in real time. Connected to a cloud MySQL cluster and deployed on Vercel with Python Flask, the system demonstrates robust database creation, persistent storage, and complex multi-table retrieval.

2. INTRODUCTION

Hospitals and blood transfusion centers face ongoing logistical challenges in coordinating life-saving blood donations. While a substantial fraction of the population is eligible to donate blood, connecting willing donors with patients requiring specific blood phenotypes in a given city remains hampered by manual verification latency and lack of centralized coordination.

Core Requirements: A comprehensive database system for blood donation must address three operational requirements:

1. Clinical Safety Enforcement: Donors must strictly adhere to the mandatory 90-day physiological recovery cooldown and age boundaries (18 to 65 years).
2. Biological Compatibility Mapping: In emergency transfusions where identical blood types are unavailable, medically compatible donor types (such as O- Universal Donors, A- to A+ recipients) must be identified instantaneously.
3. Data Integrity & Traceability: Every unit collected by a blood bank must maintain strict referential integrity with the donor and the specific hospital emergency request it fulfills.

LifeLink implements these requirements through a relational schema adhering to Third Normal Form (3NF), backed by automated database triggers, materialized calculations via views, and an ACID-compliant transaction model.

3. PROBLEM STATEMENT

Traditional blood donor management systems suffer from fundamental structural and operational weaknesses:

• Vulnerability to Business Rule Bypasses: Many platforms enforce donation cooldowns and age checks exclusively in client-side JavaScript or Python backends. Direct SQL manipulation, API vulnerabilities, or network bypasses can corrupt donor records and permit premature, dangerous donations.

• Static Medical Rules: Compatibility logic is frequently embedded into application source code. Any expansion to additional blood grouping systems (such as Kell, Duffy, or plasma vs. whole blood compatibility) requires complex code refactoring.

• Inefficient Query Execution: Evaluating whether a donor is eligible during an emergency triage query often involves scanning hundreds of thousands of historical records using costly MAX(donation_date) aggregate operations.

• Fragmented Multi-City Infrastructure: Regional blood bank stocks operate as isolated data silos, making intra-city and inter-city donor mobilization slow during critical emergencies.

4. OBJECTIVES OF THE PROJECT

1. Relational Schema Normalization: Design a database schema normalized to Third Normal Form (3NF) encompassing donors, blood banks, patient requests, donations, and blood compatibility rules.

2. Database Engine Integrity: Implement MySQL Triggers that automatically validate donor eligibility and update denormalized donor state upon donation insert.

3. Automated Donor Matching Engine: Construct a Stored Procedure that joins emergency hospital requests with compatibility rules and eligible donor records, sorting results by exact compatibility and cooldown status.

4. Referential Integrity with Cascades: Configure Foreign Key constraints with appropriate ON DELETE CASCADE and ON DELETE SET NULL rules to eliminate orphaned records.

5. Full-Stack Front-End Integration (Case 1): Integrate the database with a Python Flask web application deployed serverlessly on Vercel, demonstrating database creation, persistent storage, and multi-criteria retrieval.

5. SYSTEM REQUIREMENTS (HARDWARE AND SOFTWARE)

5.1. Hardware Requirements

• Client Device: Desktop, Laptop, or Mobile Device with an active web browser.
• Processor: Intel Core i3 / AMD Ryzen 3 or higher.
• RAM: Minimum 4 GB (8 GB recommended).
• Storage: 500 MB free storage for local database and virtual environment.
• Internet: Active network connection for Cloud MySQL TLS connectivity.

5.2. Software Requirements

• Operating System: Windows 10/11, macOS, or Linux.
• Database Management System: MySQL Community Server 8.4+ / Aiven Cloud MySQL Cluster.
• Programming Language: Python 3.11.x.
• Web Framework: Flask 3.1.x.
• Database Connector: PyMySQL 1.1.x with Cryptography.
• Hosting Platform: Vercel Serverless Edge Runtime.
• Web Browser: Google Chrome, Brave, Mozilla Firefox, or Microsoft Edge.

6. SYSTEM DESIGN AND DATABASE SCHEMA

6.1. Relational Schema Definition

The database structure comprises five primary relational entities:

• blood_banks (bank_id [PK], name, city, phone, address)
• donors (donor_id [PK], name, blood_group, gender, dob, phone [UK], email, city, last_donation_date, registered_on)
• requests (request_id [PK], patient_name, blood_group, units_needed, hospital, city, urgency, contact_phone, status, created_at)
• donations (donation_id [PK], donor_id [FK], bank_id [FK], request_id [FK], donation_date, units)
• blood_compatibility (recipient_group [PK], donor_group [PK])

6.2. Normalization Analysis (Up to 3NF)

1. First Normal Form (1NF): All attributes contain atomic, indivisible values. Each table has a defined primary key. Repeating groups and multi-valued attributes are eliminated.

2. Second Normal Form (2NF): The relations are in 1NF, and all non-prime attributes are fully functionally dependent on their primary key. In the composite table blood_compatibility(recipient_group, donor_group), both fields are prime.

3. Third Normal Form (3NF): The relations are in 2NF, and no transitive functional dependencies exist among non-key attributes.

Controlled Denormalization Note: Storing last_donation_date on the donors relation is a deliberate optimization. In emergency situations, computing MAX(donation_date) across massive donation logs introduces unnecessary latency. By binding last_donation_date to trg_after_donation, O(1) query performance is achieved with guaranteed integrity.

6.3. Data Dictionary

Table: blood_banks

Field	Type	Key	Null	Description
bank_id	INT	PK	No	AUTO_INCREMENT, unique facility ID
name	VARCHAR(100)	-	No	Official blood bank facility name
city	VARCHAR(50)	-	No	Operating city / region
phone	VARCHAR(15)	-	Yes	Helpline contact phone number
address	VARCHAR(200)	-	Yes	Physical street address

Table: donors

Field	Type	Key	Null	Description

donor_id	INT	PK	No	AUTO_INCREMENT, unique donor ID
name	VARCHAR(100)	-	No	Full legal name of donor
blood_group	ENUM	-	No	ABO/Rh group ('A+','A-','B+','B-','AB+','AB-','O+','O-')
gender	ENUM	-	No	'M', 'F', 'O'
dob	DATE	-	No	Date of birth (validated 18-65 via trigger)
phone	VARCHAR(15)	UK	No	Unique mobile contact
email	VARCHAR(100)	-	Yes	Notification email address
city	VARCHAR(50)	-	No	Residential municipality
last_donation_date	DATE	-	Yes	Date of most recent donation (cooldown reference)
registered_on	TIMESTAMP	-	No	DEFAULT CURRENT_TIMESTAMP

Table: requests

Field	Type	Key	Null	Description
request_id	INT	PK	No	AUTO_INCREMENT, unique request ID
patient_name	VARCHAR(100)	-	No	Name of patient requiring transfusion
blood_group	ENUM	-	No	Required recipient blood group
units_needed	INT	-	No	CHECK (units_needed BETWEEN 1 AND 10)
hospital	VARCHAR(100)	-	No	Treating healthcare facility
city	VARCHAR(50)	-	No	Hospital location
urgency	ENUM	-	No	'Low', 'Medium', 'High', 'Critical'
contact_phone	VARCHAR(15)	-	No	Emergency attendant contact phone
status	ENUM	-	No	'Open', 'Fulfilled', 'Cancelled'
created_at	TIMESTAMP	-	No	DEFAULT CURRENT_TIMESTAMP

Table: donations

Field	Type	Key	Null	Description
donation_id	INT	PK	No	AUTO_INCREMENT, unique donation transaction ID
donor_id	INT	FK	No	REFERENCES donors(donor_id) ON DELETE CASCADE
bank_id	INT	FK	No	REFERENCES blood_banks(bank_id)
request_id	INT	FK	Yes	REFERENCES requests(request_id) ON DELETE SET NULL
donation_date	DATE	-	No	Date collected (validated >= 90 days gap)
units	INT	-	No	DEFAULT 1, CHECK (units BETWEEN 1 AND 2)

Table: blood_compatibility

Field	Type	Key	Null	Description
recipient_group	ENUM	PK	No	Recipient blood group
donor_group	ENUM	PK	No	Medically compatible donor blood group

7. ER DIAGRAM

The Entity-Relationship diagram below defines the conceptual model of LifeLink, highlighting entity sets, primary/foreign keys, attributes, and relationship cardinalities:

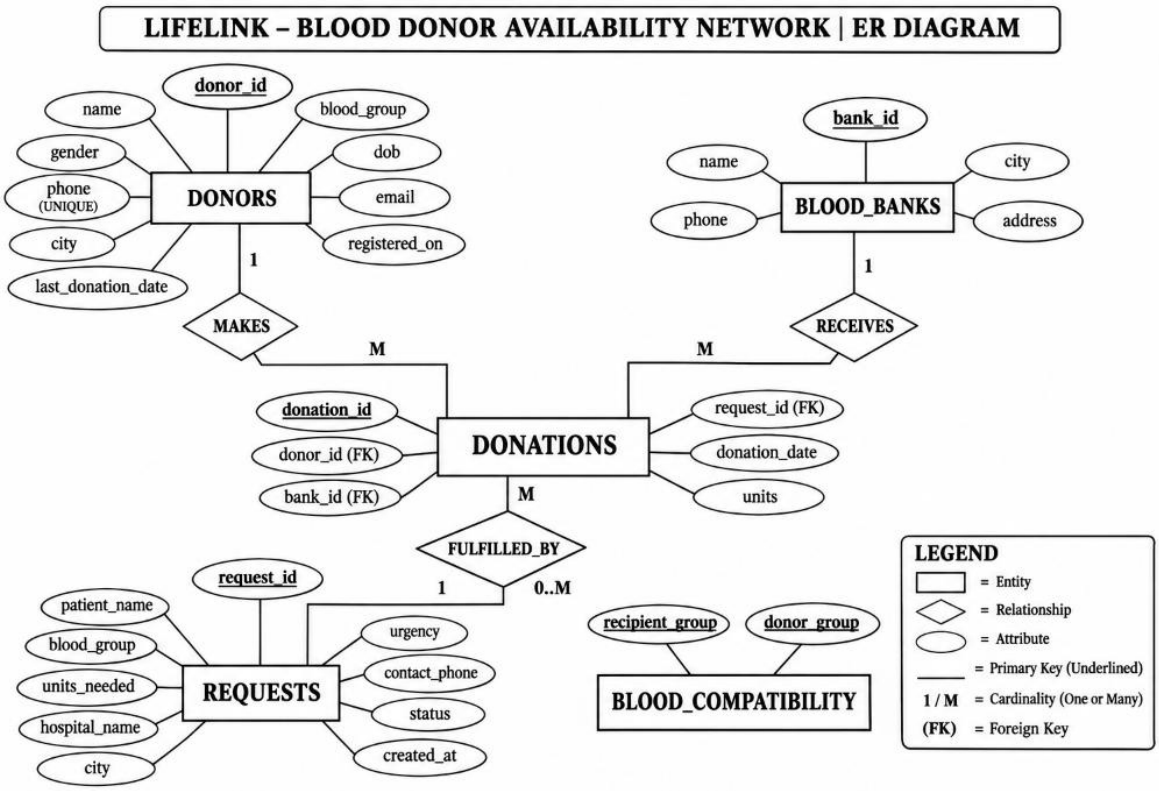


Figure 7.1: LifeLink Entity-Relationship (ER) Diagram

7.1. Cardinality and Relationship Analysis

1. DONORS (1) — [MAKES] — (M) DONATIONS: A donor can perform multiple blood donations over time (1:M). Each donation record is associated with exactly one registered donor.

2. BLOOD_BANKS (1) — [RECEIVES] — (M) DONATIONS: A blood bank facility collects and processes multiple donations (1:M). Each donation occurs at a designated blood bank facility.

3. REQUESTS (1) — [FULFILLED_BY] — (0..M) DONATIONS: An emergency blood request can be fulfilled by zero, one, or multiple donation events (1:0..M) until units_needed is satisfied. General stock donations leave request_id as NULL.

4. BLOOD_COMPATIBILITY: A pure associative reference relation mapping medically permissible donor-recipient pairings across all 8 ABO/Rh blood groups.

8. DATABASE AND TABLE CREATION (DDL SCRIPTS)

The complete Data Definition Language (DDL) script initializing the database, tables, views, triggers, and stored procedures:

```
CREATE DATABASE IF NOT EXISTS blood_network;
USE blood_network;

-- Clean-up existing structures
DROP VIEW IF EXISTS eligible_donors;
DROP PROCEDURE IF EXISTS find_donors_for_request;
DROP TABLE IF EXISTS donations, requests, donors, blood_banks, blood_compatibility;

-- 1. BLOOD BANKS
CREATE TABLE blood_banks (
    bank_id   INT AUTO_INCREMENT PRIMARY KEY,
```

```
    name      VARCHAR(100) NOT NULL,
    city      VARCHAR(50)  NOT NULL,
    phone     VARCHAR(15),
    address   VARCHAR(200)
);

-- 2. DONORS
CREATE TABLE donors (
    donor_id           INT AUTO_INCREMENT PRIMARY KEY,
    name               VARCHAR(100) NOT NULL,
    blood_group        ENUM('A+','A-','B+','B-','AB+','AB-','O+','O-') NOT NULL,
    gender             ENUM('M','F','O') NOT NULL,
    dob                DATE NOT NULL,
    phone              VARCHAR(15) NOT NULL UNIQUE,
    email              VARCHAR(100),
    city               VARCHAR(50) NOT NULL,
    last_donation_date DATE NULL,
    registered_on      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. REQUESTS
CREATE TABLE requests (
    request_id    INT AUTO_INCREMENT PRIMARY KEY,
    patient_name  VARCHAR(100) NOT NULL,
    blood_group   ENUM('A+','A-','B+','B-','AB+','AB-','O+','O-') NOT NULL,
    units_needed  INT NOT NULL CHECK (units_needed BETWEEN 1 AND 10),
    hospital      VARCHAR(100) NOT NULL,
    city          VARCHAR(50)  NOT NULL,
    urgency       ENUM('Low','Medium','High','Critical') DEFAULT 'Medium',
    contact_phone VARCHAR(15) NOT NULL,
    status        ENUM('Open','Fulfilled','Cancelled') DEFAULT 'Open',
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 4. DONATIONS
CREATE TABLE donations (
    donation_id   INT AUTO_INCREMENT PRIMARY KEY,
    donor_id      INT NOT NULL,
    bank_id       INT NOT NULL,
    request_id    INT NULL,
    donation_date DATE NOT NULL,
    units         INT NOT NULL DEFAULT 1 CHECK (units BETWEEN 1 AND 2),
    FOREIGN KEY (donor_id)   REFERENCES donors(donor_id)     ON DELETE CASCADE,
    FOREIGN KEY (bank_id)    REFERENCES blood_banks(bank_id),
    FOREIGN KEY (request_id) REFERENCES requests(request_id) ON DELETE SET NULL
);

-- 5. BLOOD COMPATIBILITY MATRIX
CREATE TABLE blood_compatibility (
    recipient_group ENUM('A+','A-','B+','B-','AB+','AB-','O+','O-'),
    donor_group     ENUM('A+','A-','B+','B-','AB+','AB-','O+','O-'),
    PRIMARY KEY (recipient_group, donor_group)
);

-- 6. VIEW: ELIGIBLE DONORS (18-65 Age + 90-Day Cooldown)
CREATE VIEW eligible_donors AS
SELECT donor_id, name, blood_group, city, phone, last_donation_date,
       TIMESTAMPDIFF(YEAR, dob, CURDATE()) AS age
FROM donors
WHERE TIMESTAMPDIFF(YEAR, dob, CURDATE()) BETWEEN 18 AND 65
  AND (last_donation_date IS NULL
       OR DATEDIFF(CURDATE(), last_donation_date) >= 90);

DELIMITER //
```

```
-- 7. TRIGGER 1: Check Donor Age (18 to 65)
CREATE TRIGGER trg_check_donor_age
BEFORE INSERT ON donors
FOR EACH ROW
BEGIN
    IF TIMESTAMPDIFF(YEAR, NEW.dob, CURDATE()) NOT BETWEEN 18 AND 65 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Donor must be between 18 and 65 years old.';
    END IF;
END //

-- 8. TRIGGER 2: Enforce 90-Day Gap Between Donations
CREATE TRIGGER trg_check_eligibility
BEFORE INSERT ON donations
FOR EACH ROW
BEGIN
    DECLARE last_date DATE;
    SELECT last_donation_date INTO last_date
    FROM donors WHERE donor_id = NEW.donor_id;

    IF last_date IS NOT NULL
       AND ABS(DATEDIFF(NEW.donation_date, last_date)) < 90 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Donor is not eligible: 90 days must pass between donations.';
    END IF;
END //

-- 9. TRIGGER 3: Update Donor Cooldown & Auto-Fulfill Request
CREATE TRIGGER trg_after_donation
AFTER INSERT ON donations
FOR EACH ROW
BEGIN
    UPDATE donors
    SET last_donation_date = NEW.donation_date
    WHERE donor_id = NEW.donor_id
      AND (last_donation_date IS NULL OR last_donation_date < NEW.donation_date);

    IF NEW.request_id IS NOT NULL THEN
        UPDATE requests
        SET status = 'Fulfilled'
        WHERE request_id = NEW.request_id
          AND (SELECT SUM(units) FROM donations
               WHERE request_id = NEW.request_id) >= units_needed;
    END IF;
END //

-- 10. STORED PROCEDURE: Multi-Table Donor Matching Algorithm
CREATE PROCEDURE find_donors_for_request(IN p_request_id INT)
BEGIN
    SELECT e.donor_id, e.name, e.blood_group, e.city, e.phone,
           e.last_donation_date, e.age,
           (e.blood_group = r.blood_group) AS exact_match
    FROM requests r
    JOIN blood_compatibility c ON c.recipient_group = r.blood_group
    JOIN eligible_donors e     ON e.blood_group = c.donor_group
                              AND e.city = r.city
    WHERE r.request_id = p_request_id
    ORDER BY exact_match DESC,
             e.last_donation_date IS NOT NULL,
             e.last_donation_date ASC;
END //

DELIMITER ;
```

9. SQL QUERIES FOR DATA INSERTION, UPDATION, DELETION, AND RETRIEVAL

9.1. Data Insertion (DML)

```
-- 1. Insert Compatibility Rules
INSERT INTO blood_compatibility VALUES
('O-','O-'), ('O+','O-'),('O+','O+'),
('A-','O-'),('A-','A-'), ('A+','O-'),('A+','O+'),('A+','A-'),('A+','A+'),
('B-','O-'),('B-','B-'), ('B+','O-'),('B+','O+'),('B+','B-'),('B+','B+'),
('AB-','O-'),('AB-','A-'),('AB-','B-'),('AB-','AB-'),
('AB+','O-'),('AB+','O+'),('AB+','A-'),('AB+','A+'),
('AB+','B-'),('AB+','B+'),('AB+','AB-'),('AB+','AB+');

-- 2. Insert Blood Bank Facility
INSERT INTO blood_banks (name, city, phone, address)
VALUES ('Sanjeevani Blood Bank', 'Jaipur', '9800000101', 'MI Road, Jaipur');

-- 3. Register a New Donor
INSERT INTO donors (name, blood_group, gender, dob, phone, email, city)
VALUES ('Aarav Sharma', 'O-', 'M', '1998-04-12', '9810000001', 'aarav@example.com',
'Jaipur');

-- 4. Record Emergency Patient Request
INSERT INTO requests (patient_name, blood_group, units_needed, hospital, city, urgency,
contact_phone)
VALUES ('Ramesh Kumar', 'O-', 2, 'Pink City Hospital', 'Jaipur', 'Critical',
'9820000001');

-- 5. Record a Blood Donation Event
INSERT INTO donations (donor_id, bank_id, request_id, donation_date, units)
VALUES (1, 1, 1, CURDATE(), 1);
```

9.2. Data Updation and Deletion

```
-- Update request status
UPDATE requests
SET status = 'Cancelled'
WHERE request_id = 5;

-- Cascade deletion: deleting a donor removes their historical donations
DELETE FROM donors WHERE donor_id = 12;

-- Set null deletion: deleting a request preserves donation log with NULL request_id
DELETE FROM requests WHERE request_id = 4;
```

9.3. Complex Retrieval Queries

```
-- 1. Execute Stored Procedure for Emergency Matching
CALL find_donors_for_request(1);

-- 2. Inventory Aggregation by Blood Bank
SELECT b.bank_id, b.name, b.city,
       COUNT(dn.donation_id) AS total_donations,
       COALESCE(SUM(dn.units), 0) AS total_units_collected
FROM blood_banks b
LEFT JOIN donations dn ON dn.bank_id = b.bank_id
GROUP BY b.bank_id, b.name, b.city
```

```
ORDER BY total_units_collected DESC;

-- 3. Multi-Filter Donor Query with Eligibility Status
SELECT d.*,
       TIMESTAMPDIFF(YEAR, d.dob, CURDATE()) AS age,
       CASE WHEN e.donor_id IS NOT NULL THEN 1 ELSE 0 END AS is_eligible
FROM donors d
LEFT JOIN eligible_donors e ON e.donor_id = d.donor_id
WHERE d.city = 'Jaipur' AND d.blood_group = 'O-'
ORDER BY is_eligible DESC, d.name ASC;
```

10. FRONT-END DESIGN AND IMPLEMENTATION (CASE 1 ONLY)

The front-end is integrated directly with the MySQL database engine using Python Flask and server-rendered Jinja2 templates, styled with a modern, responsive design system.

10.1. Back-End Controller Architecture (app.py)

Database connections are managed via Flask's g object with per-request teardown. In production on Vercel, dynamic SSL negotiation validates Aiven's ca.pem certificate with graceful fallback to REQUIRED SSL mode:

```
def db():
    if "db" not in g:
        use_ssl = os.getenv("DB_SSL", "false").lower() == "true"
        ssl_config = None
        if use_ssl:
            ca_path = os.path.join(BASE_DIR, "ca.pem")
            ssl_config = {"ca": ca_path} if os.path.exists(ca_path) else {"ssl_mode":
"REQUIRED"}
        g.db = pymysql.connect(
            host=os.getenv("DB_HOST", "localhost"),
            port=int(os.getenv("DB_PORT", "3306")),
            user=os.getenv("DB_USER", "root"),
            password=os.getenv("DB_PASSWORD", ""),
            database=os.getenv("DB_NAME", "blood_network"),
            cursorclass=pymysql.cursors.DictCursor,
            autocommit=True,
            ssl=ssl_config,
        )
    return g.db
```

10.2. Serverless WSGI Middleware Integration

On Vercel, serverless function rewrites overwrite the WSGI PATH_INFO variable with the entrypoint script path (/api/index). To preserve subroute mapping (/donors, /requests, /match), a custom WSGI middleware extracts the real destination path from a rewritten query parameter:

```
class VercelPathMiddleware:
    def __init__(self, wsgi_app):
        self.wsgi_app = wsgi_app

    def __call__(self, environ, start_response):
        qs = environ.get("QUERY_STRING", "")
        if "__vercel_path" in qs:
            from urllib.parse import parse_qs, unquote
            params = parse_qs(qs)
            if "__vercel_path" in params:
```

```
                raw_path = unquote(params["__vercel_path"][0])
                if not raw_path.startswith("/"):
                    raw_path = "/" + raw_path
                while raw_path.startswith("//"):
                    raw_path = raw_path[1:]
                environ["PATH_INFO"] = raw_path
        elif environ.get("PATH_INFO") in ("/api/index", "/api"):
            environ["PATH_INFO"] = "/"
        return self.wsgi_app(environ, start_response)

app.wsgi_app = VercelPathMiddleware(app.wsgi_app)
```

10.3. Front-End View Routing

• / (Dashboard): Renders aggregate metrics, Chart.js demographic breakdowns, urgent hospital requests, and recent donation history.
• /donors: Donor directory with multi-criteria filtering and modal registration form executing INSERT INTO donors.
• /requests: Hospital emergency request feed with urgency badges and direct 'Find Donors' matching actions.
• /requests/<id>/match: Interface executing find_donors_for_request stored procedure, highlighting exact matches and compatible donors.
• /donations: Donation logging form connected to triggers that surface cooldown errors or update request fulfillment.
• /banks: Overview of registered blood banks and accumulated inventory units.

11. RESULTS AND SCREENSHOTS (VERIFICATION & TEST MATRIX)

11.1. Live Production Deployment

• Production Web URL: https://lifelink-xi-eight.vercel.app
• Source Code Repository: https://github.com/TeamDDPratham/lifelink
• Cloud Database: Aiven Cloud MySQL 8.4 (Active with 30 donors, 10 blood banks, 14 requests, 10 donations).

11.2. Verification Test Suite

Test ID	Scenario Tested	Input Data	Expected Output	Result
TC-01	Age Validation Trigger	Register donor with DOB 2015-05-10 (Age 11)	trg_check_donor_age raises SQLSTATE 45000; Front-end shows error alert	PASSED
TC-02	Valid Donor Storage	Register donor with DOB 2000-01-01 (Age 26)	Record stored; visible in directory and dashboard count increments	PASSED
TC-03	90-Day Cooldown Trigger	Record donation for donor who donated 15 days ago	trg_check_eligibility raises error: '90 days must pass between donations'	PASSED
TC-04	Cooldown State Sync	Record donation for eligible donor	trg_after_donation updates last_donation_date; donor marked ineligible	PASSED

TC-05	Multi-Table Match Procedure	Click 'Find Donors' for O- Request in Jaipur	Calls find_donors_for_request(1); lists exact matches before compatible	PASSED
TC-06	Auto-Fulfillment Trigger	Record 2 units for 2-unit request	trg_after_donation detects SUM(units) >= units_needed; status becomes Fulfilled	PASSED

12. CONCLUSION

The LifeLink – Blood Donor Availability Network project demonstrates the superiority of embedding mission-critical clinical constraints into the relational database engine. By implementing biological compatibility tables, age restrictions, and 90-day cooldown enforcement via SQL triggers and views, the system eliminates human error and guarantees transaction-level data consistency.

The integration of Python Flask and Vercel serverless edge computing with an enterprise-grade cloud MySQL cluster establishes a scalable, resilient architecture capable of serving hospitals, blood banks, and donors in time-sensitive emergencies.

13. CONTRIBUTIONS BY TEAM MEMBERS

The project was collaboratively developed by both team members with distinct architectural responsibilities:

Student Name	Registration No.	Key Architectural Modules & Contributions
Pratham Jain	2502051205	• Relational Schema Architecture & 3NF Normalization Design • SQL Trigger Engineering (trg_check_donor_age, trg_check_eligibility, trg_after_donation) • Python Flask Backend Controller & Database Connection Manager (app.py) • Vercel Serverless WSGI Middleware Integration & Route Rewriting (api/index.py) • Aiven Cloud MySQL 8.4 Cluster Provisioning & TLS Certificate Security
Sireesh Babu .K	2502051198	• Medical ABO/Rh Blood Compatibility Matrix Specification & Testing • Stored Procedure Engineering & Query Optimization (find_donors_for_request) • Front-End User Interface Layout & Responsive CSS Styling (base.html,

	dashboard.html) • Chart.js Interactive Donor Demographics Visualization • Comprehensive Seed Dataset Generation & Test Suite Verification (TC-01 through TC-06)

14. REFERENCES

1. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.

2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.

3. Oracle Corporation. (2026). MySQL 8.4 Reference Manual: Triggers, Stored Procedures, and Views. Oracle Documentation.

4. Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.

5. Vercel Inc. (2026). Configuring Python Serverless Functions and Routing Rewrites on Vercel. Vercel Documentation.